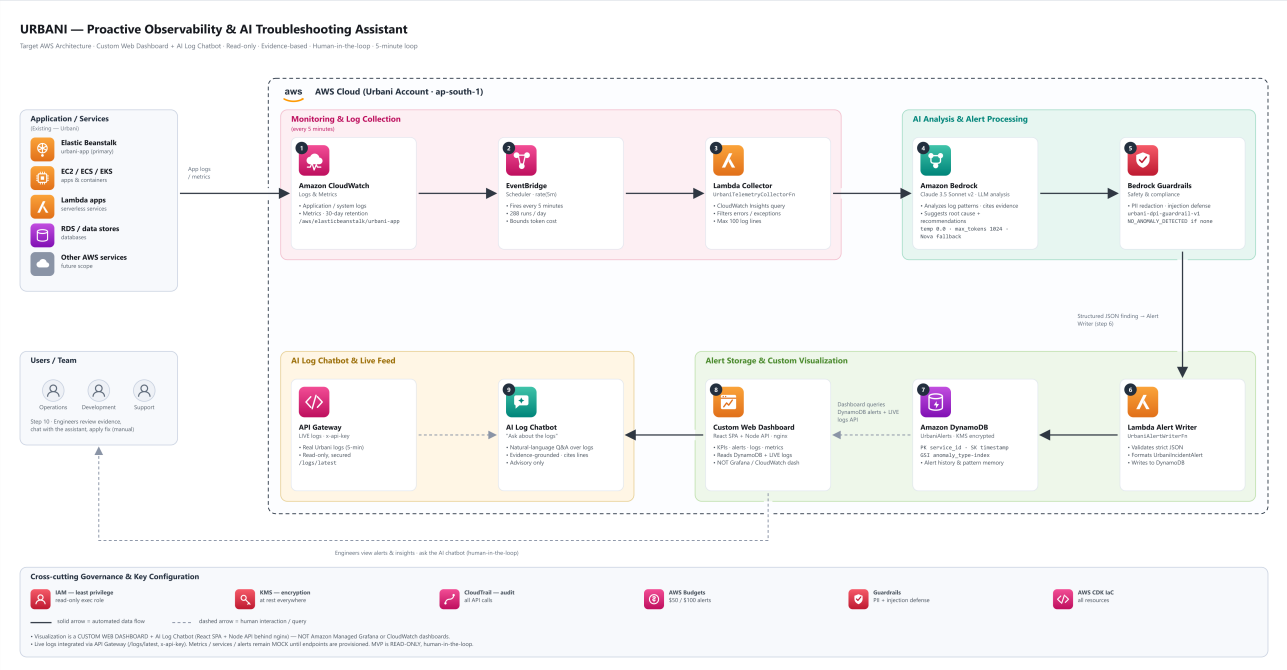
URBANI

Proactive Observability & AI Troubleshooting Assistant

Project Documentation

Complete System & Deployment Documentation

This document describes the delivered Urbani observability dashboard: its architecture, features, live AWS data integration (CloudWatch logs, AI-generated incidents, and the Amazon Bedrock-powered Copilot), the security and data-provenance model, and the ECS Fargate deployment. All content reflects the implemented system.



Target AWS architecture: ECS Fargate dashboard + serverless incident pipeline.

Table of Contents

1. **Executive Overview** 3
2. **System Architecture** 3
   - 2.1 Request flow 3
   - 2.2 Components 3
3. **Application Features** 3
   - 3.1 Dashboard & navigation 4
   - 3.2 Urbani Copilot (AI assistant) 4
4. **Live Data Integration** 4
   - 4.1 Log evidence correlation 4
   - 4.2 Incident confidence 4
5. **Security & Data Provenance** 5
   - 5.1 Authentication 5
   - 5.2 Honest provenance model 5
6. **Deployment — AWS ECS Fargate** 5
   - 6.1 Runtime topology 5
   - 6.2 Deploy sequence 6
   - 6.3 Health checks 6
7. **Operations & Maintenance** 6
8. **Quality & Testing** 6
9. **Roadmap & Next Steps** 6
10. **Glossary** 7

1. Executive Overview

The Urbani Proactive Observability & AI Troubleshooting Assistant is a custom web dashboard that gives operations teams a single, honest view of their QA service health. It surfaces live logs and AI-generated incident alerts from the Urbani QA environment and provides an in-context AI assistant (the "Urbani Copilot") that answers questions grounded in the real log and incident evidence. It is advisory and human-in-the-loop: the system never performs automated remediation.

The delivered system includes:

- **Dashboard & alerts:** health summary, alerts/incidents list and detail, logs, metrics, services, usage/cost, audit, and settings views.
- **Live AWS data:** real logs and AI-generated incidents from the Urbani QA API Gateway, with the Copilot answering via Amazon Bedrock (Nova 2 Lite) server-side.
- **Honest provenance:** every data surface is labelled LIVE, MOCK, or WAITING_FOR_INTEGRATION so viewers always know the source of what they see.
- **Production deployment:** containerized frontend and backend running as a single ECS Fargate task behind an Application Load Balancer with TLS, served at a dedicated domain.

2. System Architecture

The application is a TypeScript monorepo with three packages — a shared types library, an Express backend, and a React (Vite) single-page frontend. In production the two runnable components run as one ECS Fargate task: an nginx container serves the SPA and reverse-proxies API calls to the backend container over the shared task network.

2.1 Request flow

The browser only ever talks to the frontend origin. nginx serves the SPA at "/" and proxies "/api/*" to the backend on port 4000; the backend host is never exposed directly. The backend reads live data through a strict adapter boundary, so the data source can change without any change to the API contract, the dashboard, or business logic.

2.2 Components

Component	Technology	Role
Frontend	React 18 + Vite, served by nginx	SPA dashboard; reverse-proxies /api to the backend. The only browser-facing component (port 8080).
Backend	Node.js 20 + Express	REST API, JWT auth, live-data adapters, persistence (port 4000).
Shared	TypeScript library	Common contracts and types used by both backend and frontend.
Persistence	SQLite (better-sqlite3)	Stores users, alerts, telemetry, and audit; a single file on a mounted volume.
Incident pipeline	AWS CDK (EventBridge, Lambda, Bedrock, DynamoDB)	Serverless pipeline that generates incidents upstream (infrastructure-as-code).

3. Application Features

3.1 Dashboard & navigation

- **Dashboard:** at-a-glance service health summary and status.
- **Alerts / Incidents:** list and detail of AI-generated incidents with severity, anomaly type, confidence, evidence, and recommended actions.
- **Logs:** live log viewer with level/service/text filtering, merged from the live window and recent incident evidence.
- **Metrics, Services, Usage & Cost, Audit, Settings:** supporting operational views, each with an honest source label.

3.2 Urbani Copilot (AI assistant)

The Copilot answers operational questions ("Are there any errors right now?", "Summarize the current logs") grounded in the real log window and incident evidence. In live mode the question is sent to the Urbani QA chat endpoint, which grounds the answer server-side on Amazon Bedrock (Nova 2 Lite) across recent log windows and alerts, and returns the model answer. The assistant is advisory only and never takes action.

> **Chat vs. Logs scope**
> The Copilot is grounded server-side across both recent logs and incident alerts, so it may describe an incident that the currently-filtered log window does not list. The assistant surfaces a short grounding note making this scope explicit, so the two views are never mistaken for contradicting each other.

4. Live Data Integration

The backend integrates with the Urbani QA API Gateway (secured by an x-api-key, called server-side only). Three live data paths are wired through the adapter boundary:

Data	Upstream source	Behaviour
Logs	GET /logs/latest + /logs/history per service	Live log lines, merged latest + history, de-duplicated, newest-first, filterable.
Incidents	GET /alerts/latest + /alerts/history	AI-generated incidents; their evidence lines are correlated back to the Logs view.
Chat	POST /chat (Amazon Bedrock Nova 2 Lite)	Server-side grounded answer across recent logs + alerts; raw lines never cross the wire.
Metrics / Services	Not provisioned upstream	Served as clearly-labelled MOCK (never claimed LIVE).

4.1 Log evidence correlation

The upstream /logs endpoints currently return only "relevant" (flagged) lines, so the real log content arrives inside each incident's evidence. The backend parses that evidence — preserving original timestamps, inferring levels, and keeping message text verbatim — and surfaces it on the Logs view and in each alert's detail. This keeps the Logs view populated with genuine upstream data and gives every incident verifiable backing evidence. Nothing is ever fabricated.

4.2 Incident confidence

The upstream incident source does not report a model confidence score. Rather than show an empty value, the system derives a confidence from incident severity and labels it clearly as "(derived)" in the UI, with an explanatory tooltip, so it is never mistaken for a model-reported score.

5. Security & Data Provenance

5.1 Authentication

- **JWT auth:** all API routes except health and login require a signed JWT; passwords are bcrypt-hashed.
- **Secrets:** the JWT signing secret and the Urbani API key are injected from AWS Secrets Manager at deploy time, never baked into images or source.
- **Server-side API key:** the Urbani x-api-key is used only server-side; it is never logged, returned in responses, or sent to the browser.

5.2 Honest provenance model

Every data surface carries a source badge so viewers always know what they are looking at:

Label	Meaning
LIVE	Real data fetched from the Urbani QA environment.
MOCK	Deterministic sample data for a capability not yet provisioned upstream (e.g. metrics).
WAITING_FOR_INTEGRATION	A live source is configured but temporarily unavailable (e.g. an upstream fetch failed) — shown instead of a silent empty state.

> **No fabrication**
> The system never invents log lines or AI answers. When a live window is genuinely empty it says so; when an upstream call fails it degrades to an honest "temporarily unavailable" state. This honesty guarantee is enforced throughout the codebase.

6. Deployment — AWS ECS Fargate

The application is deployed to ECS Fargate as a single task running two containers (frontend + backend) behind an Application Load Balancer, with TLS terminated at the ALB and the dashboard served at a dedicated domain. Both container images build from the repository root and are pushed to ECR.

6.1 Runtime topology

Setting	Value
Platform	AWS ECS Fargate (serverless containers), awsvpc networking
Task size	0.5 vCPU / 1 GB (512 CPU / 1024 MiB), desired count 1
Frontend container	nginx + React SPA, port 8080 (ALB target)
Backend container	Node/Express, port 4000, reached at 127.0.0.1:4000 within the task
Load balancer	Application Load Balancer, HTTPS :443 (ACM cert), HTTP to HTTPS redirect
Persistence	SQLite at /data/urbani.sqlite on a mounted volume (single-writer; one task)
Secrets	JWT secret and Urbani API key from AWS Secrets Manager
Logs	CloudWatch Logs with per-container stream prefixes; 30-day retention

6.2 Deploy sequence

- Create ECR repositories and build + push the backend and frontend images from the repo root.
- Register the ECS task definition (two containers, secrets wired, health checks, awslogs).
- Create/attach the ALB target group and listener rules (HTTPS forward + HTTP to HTTPS redirect).
- Create or update the ECS service with deployment circuit-breaker and automatic rollback.
- Point the domain at the ALB via a Route 53 alias record.

6.3 Health checks

- **GET /health:** liveness — always 200 when the process is up (used by the container and ALB health check).
- **GET /ready:** readiness — verifies the database is reachable; returns 503 if not.
- **Frontend /healthz:** nginx liveness target for the container health check.

7. Operations & Maintenance

- **Zero-downtime deploys:** the ECS service uses a deployment circuit breaker with rollback and keeps 100% healthy capacity during deploys, so a bad image rolls back automatically.
- **Observability:** container logs stream to CloudWatch with separate frontend/backend prefixes; the backend emits structured JSON logs.
- **Configuration:** all behaviour is environment-driven; switching between mock and live data is a single INTEGRATION_MODE setting with no code change.
- **Data persistence:** the SQLite file persists on its mounted volume across task replacement; the service runs a single task because SQLite is single-writer.

> **Security action before go-live**
>
> The bootstrap admin password and the Urbani API key must be set to strong values in AWS Secrets Manager (not the development defaults), and the previously shared API key should be rotated. The JWT secret is already generated and stored in Secrets Manager by the deployment script.

8. Quality & Testing

The backend is covered by an automated test suite (unit and adapter-level) that runs as part of the build. Coverage includes the live log/alert adapters, log-evidence parsing and correlation, derived confidence, the degraded-vs-empty data distinction, authentication, and the chat grounding flow. The full build compiles the shared, backend, and frontend packages cleanly, and the frontend passes a strict TypeScript type check.

- **Automated tests:** backend suite covering adapters, services, and auth, run on every build.
- **Type safety:** end-to-end TypeScript with a strict frontend type check.
- **Honest-data tests:** specific tests assert that empty or absent upstream data never produces fabricated output.

9. Roadmap & Next Steps

Natural extensions beyond the delivered system, each fitting the existing architecture without rework:

- **DynamoDB persistence:** move from single-task SQLite to the already-provisioned DynamoDB table to allow horizontal scaling.

- **Native Bedrock analyze:** wire the full in-app Bedrock pipeline for on-demand analysis behind the existing provider interface.
- **Live metrics & services:** promote metrics and services from MOCK to LIVE once those endpoints are provisioned upstream.
- **High availability:** raise the task count across two AZs once persistence supports concurrent writers.

10. Glossary

Term	Definition
ALB	Application Load Balancer — the public entry point that terminates TLS and routes to the service.
ECS Fargate	Serverless container runtime; no servers to manage, capacity set as task CPU/ memory.
Provenance badge	The LIVE / MOCK / WAITING_FOR_INTEGRATION label shown on every data surface.
Copilot	The in-app AI assistant that answers questions grounded in live logs and incidents.
Incident evidence	The real log lines attached to an AI-generated incident, used as backing proof.
Nova 2 Lite	The Amazon Bedrock model used to generate incident analysis and Copilot answers.